

main

June 6, 2026

1 Diabetes Prediction

Team: - Brandon Cabrera

Course: CISD 43 Spring 2026

Project Description: Project involves analyzing a diabetes dataset which has key indicators for diagnosing the medical condition. The goal is to be able to train a model to accurately predict whether the individual has diabetes

```
[1]: #Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

install dependencies if you want to follow along: !pip install pandas numpy matplotlib seaborn scikit-learn

if you have uv you can run uv sync and then activate your virtual environment

2 Approach

Predicting whether someone has diabetes or not is going to be a classification problem. So I will use a KNN Classifier model in order to get this done. First I will clean the data if need be, make some visualizations to see if there are some patterns worth noting, and finally work on making an accurate model.

3 EDA

```
[2]: diabetes_data = pd.read_csv("data/diabetes.csv")
diabetes_data.head()
```

```
[2]:   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI   \
0             6     148             72             35         0  33.6
1             1      85             66             29         0  26.6
2             8     183             64              0         0  23.3
3             1      89             66             23        94  28.1
4             0     137             40             35       168  43.1
```

	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

Thankfully, the features are all numerical so we don't have to worry about encoding any categories. Let's check for any missing values.

```
[3]: diabetes_data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Pregnancies                          768 non-null    int64
1   Glucose                              768 non-null    int64
2   BloodPressure                        768 non-null    int64
3   SkinThickness                       768 non-null    int64
4   Insulin                             768 non-null    int64
5   BMI                                  768 non-null    float64
6   DiabetesPedigreeFunction             768 non-null    float64
7   Age                                  768 non-null    int64
8   Outcome                             768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

```
[4]: diabetes_data.isna().sum()
```

```
[4]: Pregnancies      0
      Glucose          0
      BloodPressure    0
      SkinThickness    0
      Insulin          0
      BMI              0
      DiabetesPedigreeFunction  0
      Age              0
      Outcome          0
      dtype: int64
```

The dataset is clean so now to do some visualizations.

```
[5]: #Code modified from an old project
      def plot_boxplot_stats(x:str) -> pd.DataFrame:
          """Plots a boxplot and returns a DataFrame containing descriptive stats shown_
          ↳by the boxplot"""
```

Args:

x (String): A column name from diabetes_data

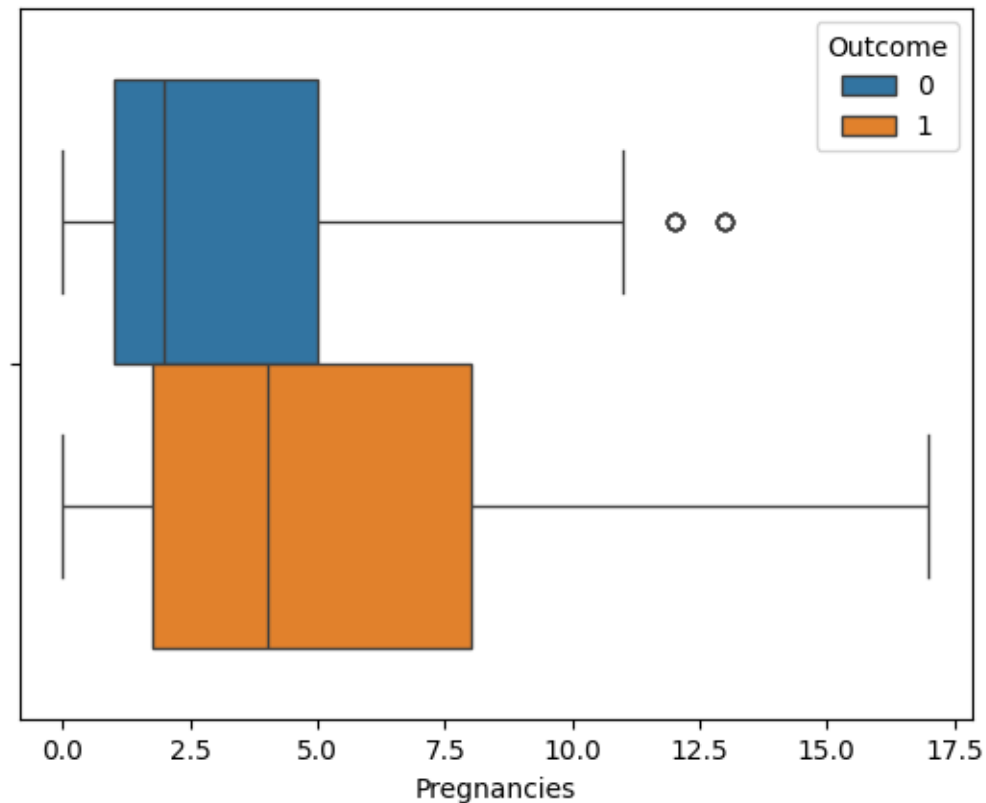
y (String): A column name from diabetes_data

Returns:

DataFrame: Contains descriptive stats of what is shown on the boxplot
"""

```
sns.boxplot(data = diabetes_data, x = x, hue = 'Outcome')
plt.show()
group_0 = diabetes_data[diabetes_data['Outcome'] == 0][x]
group_1 = diabetes_data[diabetes_data['Outcome'] == 1][x]
group_combined = pd.DataFrame({'No Diabetes' : group_0.describe(), 'With_
↳Diabetes': group_1.describe()})
return group_combined
```

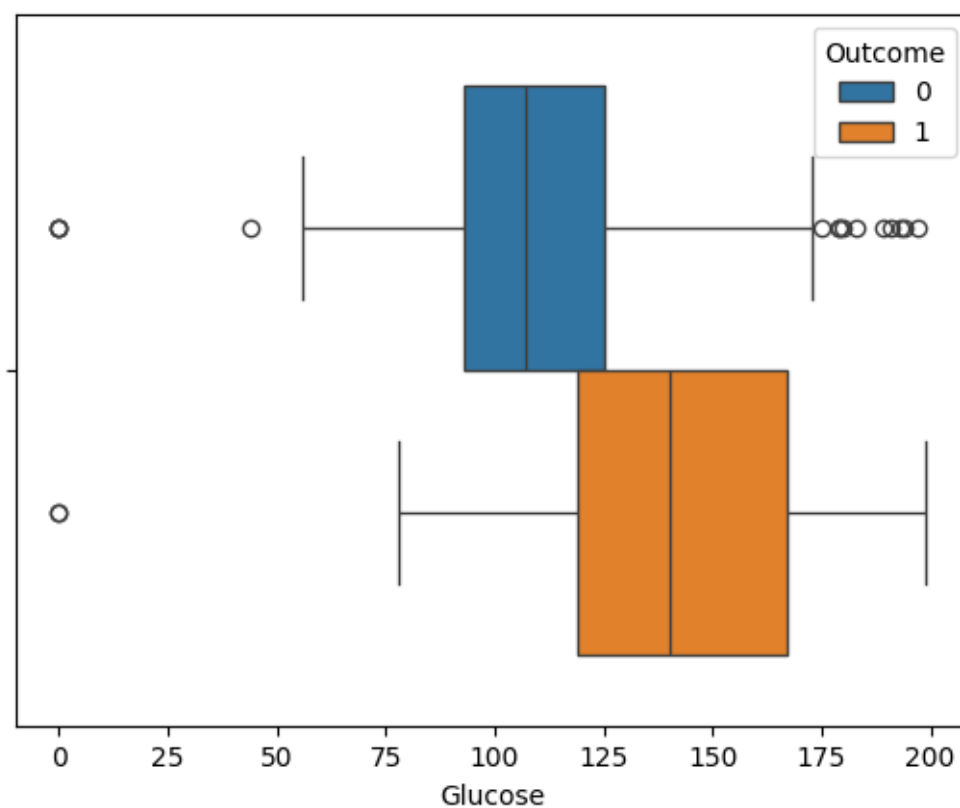
```
[6]: pregnancies_groups = plot_boxplot_stats('Pregnancies')
pregnancies_groups
```



```
[6]:
```

	No Diabetes	With Diabetes
count	500.000000	268.000000
mean	3.298000	4.865672
std	3.017185	3.741239
min	0.000000	0.000000
25%	1.000000	1.750000
50%	2.000000	4.000000
75%	5.000000	8.000000
max	13.000000	17.000000

```
[7]: glucose_groups = plot_boxplot_stats('Glucose')
      glucose_groups
```

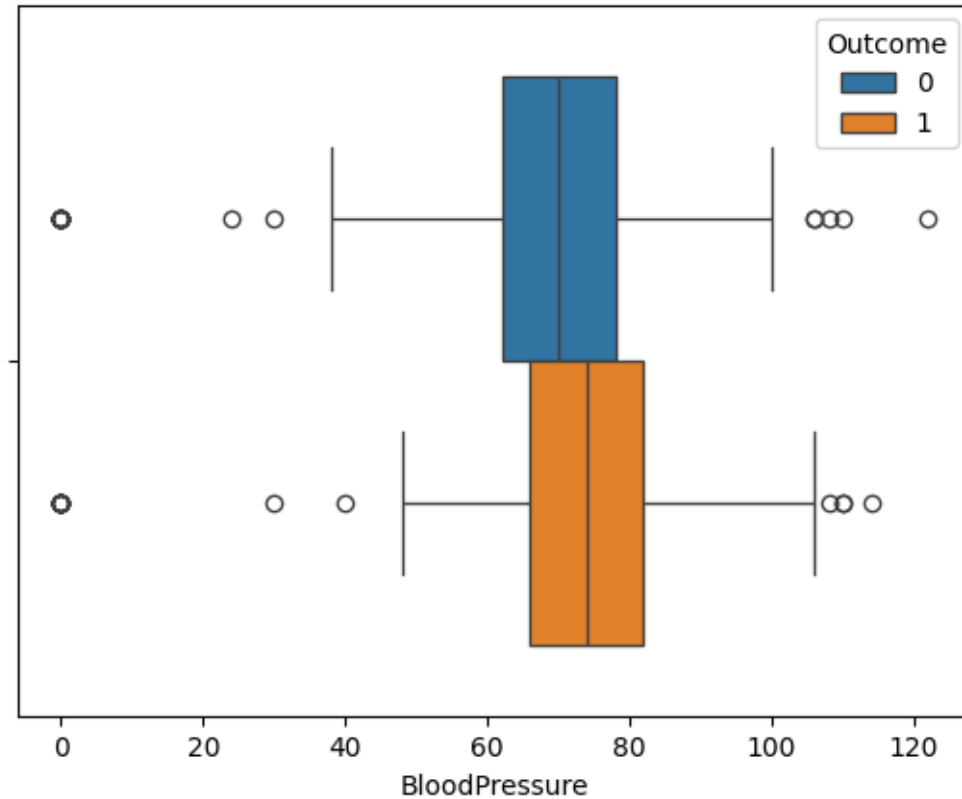


```
[7]:
```

	No Diabetes	With Diabetes
count	500.0000	268.000000
mean	109.9800	141.257463
std	26.1412	31.939622
min	0.0000	0.000000
25%	93.0000	119.000000
50%	107.0000	140.000000
75%	125.0000	167.000000

```
max      197.0000    199.000000
```

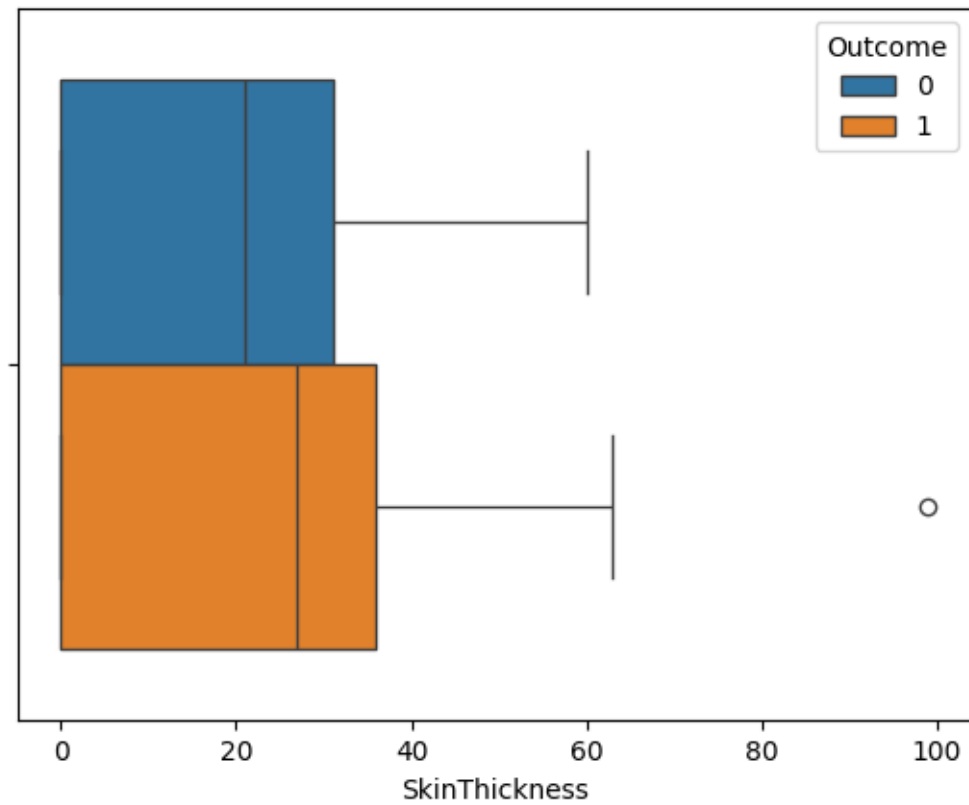
```
[8]: blood_pressure_groups = plot_boxplot_stats('BloodPressure')
      blood_pressure_groups
```



```
[8]:
```

	No Diabetes	With Diabetes
count	500.000000	268.000000
mean	68.184000	70.824627
std	18.063075	21.491812
min	0.000000	0.000000
25%	62.000000	66.000000
50%	70.000000	74.000000
75%	78.000000	82.000000
max	122.000000	114.000000

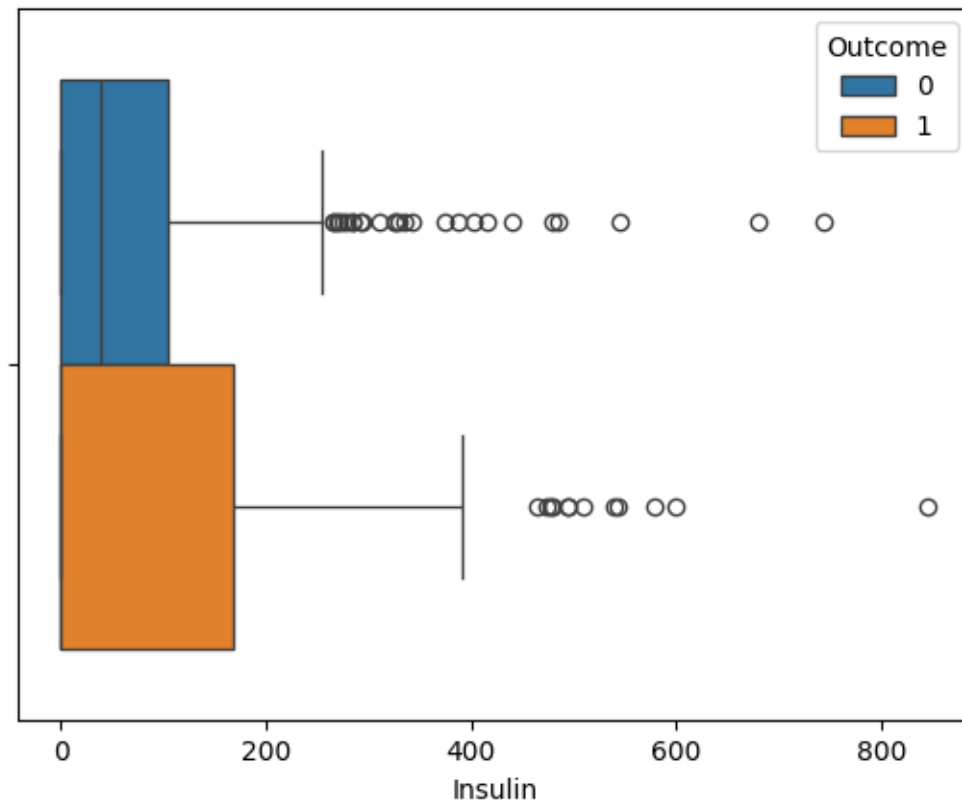
```
[9]: skin_thickness_groups = plot_boxplot_stats('SkinThickness')
      skin_thickness_groups
```



```
[9]:
```

	No Diabetes	With Diabetes
count	500.000000	268.000000
mean	19.664000	22.164179
std	14.889947	17.679711
min	0.000000	0.000000
25%	0.000000	0.000000
50%	21.000000	27.000000
75%	31.000000	36.000000
max	60.000000	99.000000

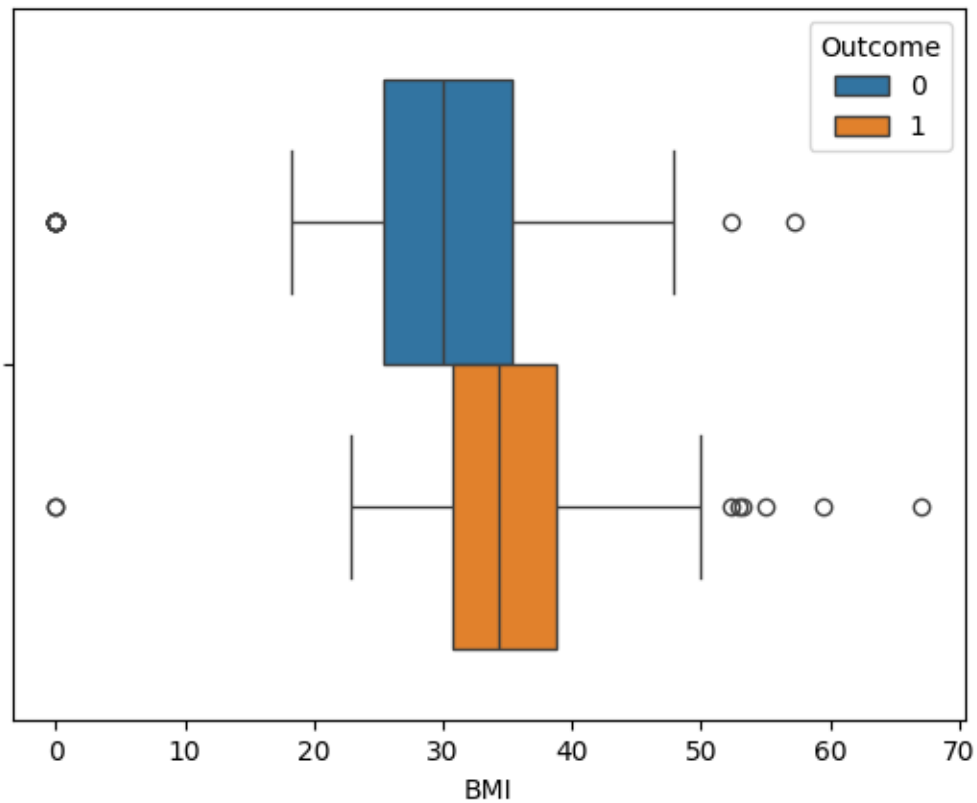
```
[10]: insulin_groups = plot_boxplot_stats('Insulin')
insulin_groups
```



```
[10]:
```

	No Diabetes	With Diabetes
count	500.000000	268.000000
mean	68.792000	100.335821
std	98.865289	138.689125
min	0.000000	0.000000
25%	0.000000	0.000000
50%	39.000000	0.000000
75%	105.000000	167.250000
max	744.000000	846.000000

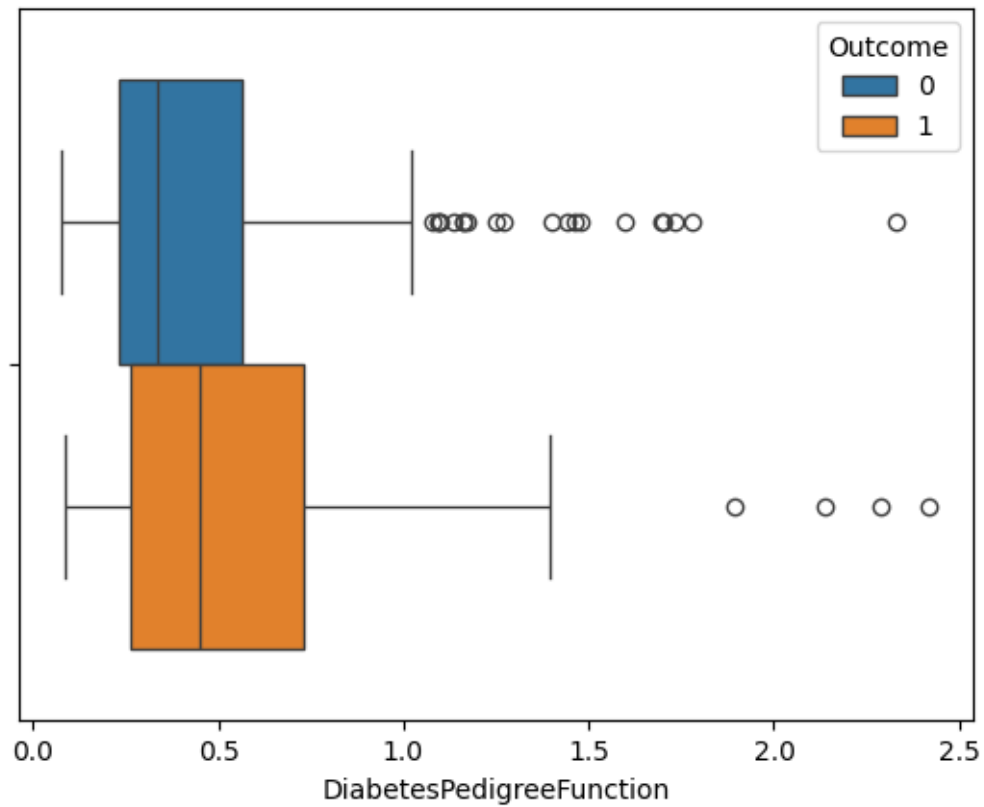
```
[11]: bmi_groups = plot_boxplot_stats('BMI')
bmi_groups
```



```
[11]:
```

	No Diabetes	With Diabetes
count	500.000000	268.000000
mean	30.304200	35.142537
std	7.689855	7.262967
min	0.000000	0.000000
25%	25.400000	30.800000
50%	30.050000	34.250000
75%	35.300000	38.775000
max	57.300000	67.100000

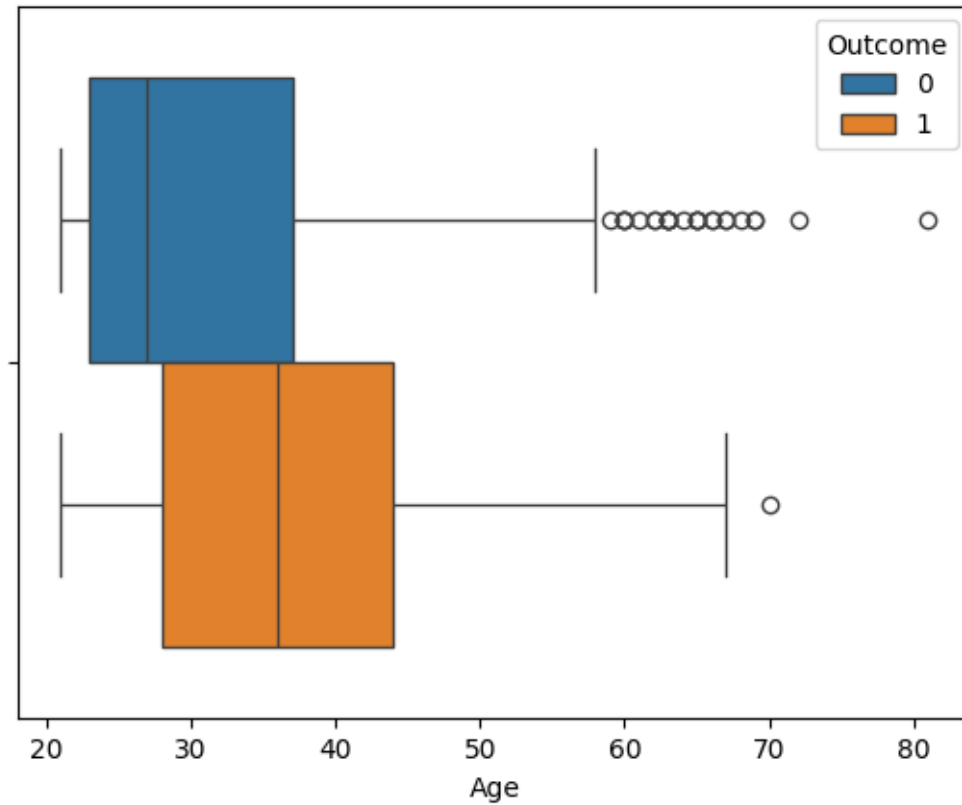
```
[12]: diabetes_pedigree_function_groups =
↳ plot_boxplot_stats('DiabetesPedigreeFunction')
diabetes_pedigree_function_groups
```

```
[12]:
```

	No Diabetes	With Diabetes
count	500.000000	268.000000
mean	0.429734	0.550500
std	0.299085	0.372354
min	0.078000	0.088000
25%	0.229750	0.262500
50%	0.336000	0.449000
75%	0.561750	0.728000
max	2.329000	2.420000

```
[13]: age_groups = plot_boxplot_stats('Age')
age_groups
```



```
[13]:
```

	No Diabetes	With Diabetes
count	500.000000	268.000000
mean	31.190000	37.067164
std	11.667655	10.968254
min	21.000000	21.000000
25%	23.000000	28.000000
50%	27.000000	36.000000
75%	37.000000	44.000000
max	81.000000	70.000000

4 Machine Learning Section

```
[14]: from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split

X = diabetes_data.iloc[:, :-1] # exclude Outcome
y = diabetes_data['Outcome'] # include only Outcome
```

```
[15]: X
```

```
[15]:
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	\
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	
..	...	...	...	...	...	...	
763	10	101	76	48	180	32.9	
764	2	122	70	27	0	36.8	
765	5	121	72	23	112	26.2	
766	1	126	60	0	0	30.1	
767	1	93	70	31	0	30.4	

	DiabetesPedigreeFunction	Age
0	0.627	50
1	0.351	31
2	0.672	32
3	0.167	21
4	2.288	33
..	...	...
763	0.171	63
764	0.340	27
765	0.245	30
766	0.349	47
767	0.315	23

[768 rows x 8 columns]

```
[16]: y
```

```
[16]:
```

0	1
1	0
2	1
3	0
4	1
..	
763	0
764	0
765	0
766	1
767	0

Name: Outcome, Length: 768, dtype: int64

```
[17]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
↳ random_state=42)
```

```
[18]: from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

knn = KNeighborsClassifier()
parameter_list = [{'n_neighbors': [3,6,9], 'weights': ['uniform', 'distance']}]
grid_search_cv = GridSearchCV(knn, parameter_list, cv = 5)
grid_search_cv.fit(X_train_scaled, y_train)
knn_model = grid_search_cv.best_estimator_
knn_model
```

```
[18]: KNeighborsClassifier(n_neighbors=6, weights='distance')
```

```
[19]: from sklearn.metrics import classification_report, confusion_matrix
preds = knn_model.predict(X_test_scaled)
print(classification_report(y_test, preds, target_names=['No Diabetes', 'Has_
↳Diabetes']))
tn, fp, fn, tp = confusion_matrix(y_test, preds).ravel().tolist()
tn, fp, fn, tp
```

	precision	recall	f1-score	support
No Diabetes	0.75	0.78	0.76	151
Has Diabetes	0.55	0.50	0.52	80
accuracy			0.68	231
macro avg	0.65	0.64	0.64	231
weighted avg	0.68	0.68	0.68	231

```
[19]: (118, 33, 40, 40)
```

5 Conclusions

From the boxplots and the descriptive stats of the two groups: people with diabetes and people without diabetes; we can tell that people with diabetes tend to have higher numbers for health metrics. So metrics like insulin, BMI, number of pregnancies, and skin thickness were higher than people who didn't have diabetes. The dataset doesn't distinguish which type of diabetes is present, type 1 people usually will have lower levels of insulin than people without diabetes. For type 2 diabetes it's the other way around since they produce too much of it.

Looking at the classification report of the KNN model's performance. For the precision score we can see that the model had a harder time accurately predicting if the patient had diabetes based on the features. Looking at the accuracy the model tends to predict correctly more than it does incorrectly with an accuracy of 0.68. According to the confusion matrix: there was 118 true negative, 33 false positives, 40 false negatives, and 40 true positives.

6 Credits

Looked at the MongoDB portion of previous year's submissions and adapted it to fit my data <https://github.com/lukalee/CISD-43-Big-Data-Modeling-Analysis/tree/11f9e916a3e0fed154f3a4ecec6934654b9dd6f3>